Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Master Thesis – Structure Proposal B

Template B: A Research-Question-Driven Empirical Study

by

Author's Name

7th October, 2026

University of Kaiserslautern-Landau
Department of Computer Science
Distributed Systems Lab

Examiner: Prof. Dr.-Ing. Jens B. Schmitt
M. Sc. Supervisor Name

About This Proposal

Organising idea. The thesis is a measurement study. The system is described only as much as the reader needs, and the body has one chapter per research question, each ending with an explicit answer. This is the convention of empirical software engineering, with a study-design chapter and categorised threats to validity [1]. Systems characterisation papers use the same shape, with one section per factor studied [2].

Choose it if the main contribution is the answers: what deception at the edge costs, and whether it breaks anything. The three questions the benchmark was built around (performance, latency, functionality; `BENCHMARK_METHODOLOGY.md` §6) map directly onto chapters, and a fourth asks how much depends on the runtime.

Trade-off. Design and implementation shrink to one chapter (*The System under Study*). A reader looking for the engineering has to find it there or in the appendix.

Ch.	Title	approx. pages
1	Introduction	5
2	Background and Related Work	12
3	The System under Study	10
4	Study Design	12
5	RQ1: Latency Added	8
6	RQ2: CPU and Capacity	12
7	RQ3: Functional Correctness	8
8	RQ4: Influence of the Extension Runtime	8
9	Discussion	7
10	Conclusion	3
A–D	Appendices	as needed

Page counts are a rough budget, not a department rule.

How to read the rest. The table of contents is the proposed structure. Under each heading is a short note on what to write there, and a *Material* line pointing to the place in the repository that already holds it (`results/` and `plots/` are short for `benchmarks/results/` and `benchmarks/results/plots/`).

Front matter (unchanged from the department template)

- **Title page** and **Eigenständigkeitserklärung**, as generated by `rptuma.cls`. The declaration requires disclosing generative-AI use beyond grammar and style.
- **Abstract** (English): the problem (deception at the web edge has an unknown cost), the approach (one mechanism on four edges, levelled to identical behaviour), the method (paired benchmark on three measurement planes) and the headline numbers.
- **Zusammenfassung**: the same abstract in German.
- **Contents**, **List of Figures**, **List of Tables**.
- **Glossary** (optional, the class's `glossar` environment): WADM, honeytoken, edge, tier, VU, iteration, SLO, cgroup, LuaJIT, PUC Lua, proxy-wasm.

In the department template's own example, websites are cited through an archived copy (`web.archive.org`); do the same for any web source in the list at the end.

Contents

1 Introduction **1**
- 1.1 Context . . . 1
- 1.2 Research Gap . . . 1
- 1.3 Research Questions . . . 1
- 1.4 Contributions . . . 1
- 1.5 Outline . . . 1

2 Background and Related Work **2**
- 2.1 Honeytokens and Application-Layer Deception . . . 2
- 2.2 Edge Proxies and Their Extension Runtimes . . . 2
- 2.3 Measuring Proxy Overhead . . . 2
- 2.4 Benchmarking Pitfalls . . . 2
- 2.5 Research Gap Revisited . . . 2

3 The System under Study **3**
- 3.1 WADM in Brief . . . 3
- 3.2 The Four Implementations . . . 3
- 3.3 Equivalence as a Precondition for Comparison . . . 3

4 Study Design **4**
- 4.1 Variables . . . 4
- 4.2 Environment . . . 4
- 4.3 Workload and Load Model . . . 4
- 4.4 Instruments . . . 4
  - 4.4.1 In-Edge Timers . . . 4
  - 4.4.2 End-to-End Latency . . . 4
  - 4.4.3 CPU per Request . . . 5
- 4.5 Paired Design and Replication . . . 5
- 4.6 Validity Gates and Negative Control . . . 5
- 4.7 Analysis Procedure . . . 5
- 4.8 Lessons from Earlier Harness Versions . . . 5

5 RQ1: Latency Added **6**
- 5.1 Median Latency per GET across Rates . . . 6
- 5.2 Tail Latency . . . 6
- 5.3 Request Position and Reconnects . . . 6

5.4 Answer to RQ1 . . . 6

6 RQ2: CPU and Capacity 7
6.1 CPU per Request . . . 7
6.2 Cost Attribution . . . 7
6.3 The Probe Effect . . . 7
6.4 Per-Mechanism Cost and Page Size . . . 7
6.5 Capacity and the Binding Resource . . . 7
6.6 Answer to RQ2 . . . 8

7 RQ3: Functional Correctness 9
7.1 Response Integrity under Load . . . 9
7.2 Detection Recall and False Positives . . . 9
7.3 Response-Shape Matrix . . . 9
7.4 Answer to RQ3 . . . 9

8 RQ4: Influence of the Extension Runtime 10
8.1 Per-Kind Operation Cost . . . 10
8.2 The SQLi Trap: Scan Depth versus Decoding . . . 10
8.3 Case Study: Apache mod_lua Segfaults . . . 10
8.4 Answer to RQ4 . . . 10

9 Discussion 11
9.1 Synthesis across Research Questions . . . 11
9.2 Recommendations for Operators . . . 11
9.3 Threats to Validity . . . 11
9.3.1 Construct Validity . . . 11
9.3.2 Internal Validity . . . 11
9.3.3 External Validity . . . 11
9.3.4 Conclusion Validity . . . 11

10 Conclusion 12
10.1 Summary of Answers . . . 12
10.2 Future Work . . . 12

A Configuration Reference 13

B Experiment Parameters and Run Log 14

C Full Result Tables 15

D Reproduction Guide 16

1 Introduction

1.1 Context

Edge proxies can plant honeytokens into any web application without changing its code [3]. Whether that is affordable has not been measured.

1.2 Research Gap

Deception work studies whether a lure is effective; proxy-overhead work measures generic filters. Nothing measures deception logic across edge runtimes.

1.3 Research Questions

RQ1 How much latency does WADM add per request on each edge, and how does that change with load? **RQ2** What does it cost in CPU and capacity, and where does that cost arise? **RQ3** Does it break what the server did without it, and does it detect what it should? **RQ4** How much of the answer depends on the extension runtime (LuaJIT, PUC Lua, WebAssembly)?

1.4 Contributions

The study design, the four levelled implementations, the measured answers, and a public harness.

1.5 Outline

One line per chapter, naming which research question each answers.

2 Background and Related Work

Background and related work combined, as is usual for a study of this kind; each section ends with how this thesis differs.

2.1 Honeytokens and Application-Layer Deception

From honeypots to honeytokens and deceptive web artefacts, and how they are deployed without changing code [4, 5, 6, 7, 3]

2.2 Edge Proxies and Their Extension Runtimes

Request phases, body buffering, and the runtimes compared: LuaJIT, PUC Lua 5.1, V8 WebAssembly.

2.3 Measuring Proxy Overhead

Service-mesh sidecar studies [2].

2.4 Benchmarking Pitfalls

Known flaws [8, 9] and coordinated omission; these motivate Chapter 4.

2.5 Research Gap Revisited

Restate the gap against the work just reviewed, and show why each RQ addresses part of it.

3 The System under Study

Just enough of WADM to interpret the measurements.

3.1 WADM in Brief

Five honeytoken kinds plus the fake SQL-injection trap, all driven by one `config.json`. One table: kind, where it is planted, what triggers it.

Material: `CONTEXT.md`, `config.json`

3.2 The Four Implementations

A table: edge, runtime, hooks used, how bodies are buffered, what differs.

Material: `nginx/README.md`, `envoy_scripts/README.md`, `wasm-filter/README.md`, `apache_scripts/README.md`

3.3 Equivalence as a Precondition for Comparison

The canonical mechanism, tuning parity (keep-alive, logging, code caching), and the parity check that proves the four behave identically.

Material: `docs/EDGE_LEVELING.md`, `benchmarks/parity_check.py`

4 Study Design

The experiment as a reproducible procedure.

4.1 Variables

Independent: edge, tier, arrival rate, request mix, page size. Dependent: latency difference, CPU per request, capacity, share of intact responses, alert recall.

4.2 Environment

VM, cpuset pinning (system, edge, origin, load generator), guest tuning, image versions.

Material: `docs/BENCHMARK_METHODOLOGY.md` §4

4.3 Workload and Load Model

An open-loop arrival rate of 1, 10 or 100 iterations/s; nine requests per iteration; probes in Williams order [10].

Material: `benchmarks/test.js`

4.4 Instruments

4.4.1 In-Edge Timers

Wall-clock regions around detection and injection, logged in µs.

4.4.2 End-to-End Latency

The k6 summary, per probe.

4.4.3 CPU per Request

A host-side sampler of cgroup `cpu.stat`, integrated over the measured window.

4.5 Paired Design and Replication

Bare, silent and instrumented tiers in balanced order, 5 replicates. silent − bare is the cost of WADM; instrumented − silent is the cost of measuring it.

4.6 Validity Gates and Negative Control

The `assess_run()` thresholds; origin CPU as a negative control; detection of host freezes.

Material: `benchmarks/wadm_timings.py`

4.7 Analysis Procedure

Medians of paired differences, the range over replicates, "resolved" when the replicates agree in sign, and a trimmed mean for the timers.

4.8 Lessons from Earlier Harness Versions

Briefly, why the design looks like this: position confounding, the mean dominated by a tail, unpaired tiers, log writes in the measured path, the load generator competing with the edge.

Material: `docs/BENCHMARK_METHODOLOGY.md` §1

5 RQ1: Latency Added

5.1 Median Latency per GET across Rates

+109 to +218 µs (13–23%), resolved in every `GET` cell.

Material: `results/paired_headline.md`, `plots/paired_latency_added.png`

5.2 Tail Latency

p95 grows more than the median on every edge. Apache at 100 iterations/s collapses.

5.3 Request Position and Reconnects

The first request after the think-time sleep pays a wake-up cost. Apache closes the connection after the trap's 500.

Material: `docs/THESIS_NOTES.md` (2026-09-21, §5b), `plots/paired_latency_shift.png`

5.4 Answer to RQ1

One paragraph: the number per edge, how it changes with load, and how certain it is.

6 RQ2: CPU and Capacity

6.1 CPU per Request

CPU added per request: OpenResty +33, Envoy+Lua +41, WASM +123, Apache +200 µs.

6.2 Cost Attribution

The in-edge timers cover 5–22% of that CPU; the rest is VM entry and exit, body filters and log writes.

Material: `plots/cost_attribution.png`

6.3 The Probe Effect

The instrumentation costs as much as WADM itself on two edges.

6.4 Per-Mechanism Cost and Page Size

The `GET` path against the trap; injection grows linearly up to 768 KiB.

Material: `results/mechanisms.md`, `plots/size_sweep.png`

6.5 Capacity and the Binding Resource

Share of bare capacity kept with silent WADM: OpenResty 71%, WASM 71%, Envoy+Lua 84%, Apache 30%, each reported with the resource that limited it.

Material: `results/capacity_summary.md`

6.6 Answer to RQ2

CPU and capacity per edge, and where the cost arises.

7 RQ3: Functional Correctness

7.1 Response Integrity under Load

Share of intact responses per probe and rate. Apache is the only edge that falls below 100%.

Material: `results/paired_functional.md`

7.2 Detection Recall and False Positives

`WADM ALERT` and `WADM TRAP` lines counted against expected; zero on bare.

7.3 Response-Shape Matrix

Gzip-encoded bodies and range requests are corrupted on all four edges. HEAD, bodies over 1 MiB and ETags each fail on some of them.

Material: `results/functional_matrix.md`, `benchmarks/functional_matrix.py`

7.4 Answer to RQ3

What breaks, on which edge, and whether detection is complete.

8 RQ4: Influence of the Extension Runtime

8.1 Per-Kind Operation Cost

In-edge timer distributions per honeytoken kind and edge.

Material: `plots/paired_internal_timers.png`

8.2 The SQLi Trap: Scan Depth versus Decoding

In the 2×2, decoding costs 0.5–2.5 µs on the Lua edges and nothing on WASM; scan depth is visible only on PUC Lua.

Material: `docs/THESIS_NOTES.md` (2026-09-21), `results/sqli.md`

8.3 Case Study: Apache mod_lua Segfaults

Crashes driven by cumulative requests, and the `LuaScope conn` trade-off.

Material: `docs/BENCHMARK_METHODOLOGY.md` §4b–4c

8.4 Answer to RQ4

How much of RQ1–RQ3 is explained by the runtime rather than by the deception logic.

9 Discussion

9.1 Synthesis across Research Questions

One table of edges against the four answers.

9.2 Recommendations for Operators

Which edge to use, and which response types to exclude from injection.

9.3 Threats to Validity

9.3.1 Construct Validity

Whether silent − bare really is "the cost of WADM"; the timer regions.

9.3.2 Internal Validity

Host freezes, position effects, warm-up, load-generator contention.

9.3.3 External Validity

One VM, synthetic traffic, a static origin, no TLS.

9.3.4 Conclusion Validity

Five replicates, ranges rather than significance tests, resolution limits.

10 Conclusion

10.1 Summary of Answers

RQ1–RQ4 in one sentence each.

10.2 Future Work

POST-body inspection on all four edges, compressed bodies, real attacker traffic, TLS and HTTP/2.

A Configuration Reference

Every `config.json` field per honeytoken kind, and the SQLi trap's fields.

Material: `CONTEXT.md`

B Experiment Parameters and Run Log

Suite IDs, rejected cells and their reasons, incidents such as the host sleeping mid-run.

Material: `docs/BENCHMARK_METHODOLOGY.md` §8

C Full Result Tables

Per-rate tables and figures.

Material: `benchmarks/results/plots/by_rate/`

D Reproduction Guide

Commands and machine configuration, with how long each step takes.

Material: `docs/BENCHMARK_METHODOLOGY.md` §3–4

Sources Used for This Proposal

[1] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[2] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz, D. Zhuo, and R. Mahajan, "Dissecting service mesh overheads." arXiv:2207.00592, 2022.

[3] M. Kahlhofer and S. Rass, "Application layer cyber deception without developer interaction," in *3rd Workshop on Active Defense and Deception (ADnD), co-located with IEEE EuroS&P*, 2024. arXiv:2405.12852.

[4] L. Spitzner, "Honeytokens: The other honeypot." SecurityFocus, 2003.

[5] B. M. Bowen, S. Hershkop, A. D. Keromytis, and S. J. Stolfo, "Baiting inside attackers using decoy documents," in *Security and Privacy in Communication Networks (SecureComm)*, pp. 51–70, 2009.

[6] A. Juels and R. L. Rivest, "Honeywords: Making password-cracking detectable," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pp. 145–160, 2013.

[7] N. Virvilis, B. Vanautgaerden, and O. S. Serrano, "Changing the game: The art of deceiving sophisticated attackers," in *6th International Conference on Cyber Conflict (CyCon)*, pp. 87–97, 2014.

[8] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.

[9] G. Heiser, "Systems benchmarking crimes." `https://gernot-heiser.org/benchmarking-crimes.html`. Accessed: 07.10.2026.

[10] E. J. Williams, "Experimental designs balanced for the estimation of residual effects of treatments," *Australian Journal of Scientific Research, Series A*, vol. 2, no. 2, pp. 149–168, 1949.